Week 3 – Day 1 – Assignments

INPUT

User:

Analyze my existing code base for selenium with java implementation

Instruction

[STRICT] GO STEP BY STEP up on my approval proceed with next step

[MANDATORY] Find code logic to generate feature file and the page class with selenium and JAVA

[IMPORTANT] List the files in which implementation is done for Selenium with JAVA

[CRITICAL] Implement code logic to generate feature file and the page class with Playwright and TypeScript

[CRITICAL] List the files in which implementation is done for Playwright with TypeScript

[STRICT] Provide option to user whether they need feature file and page class for Selenium with JAVA or Playwright with TypeScript

SUMMARY OF COPILOT

GitHub Copilot: Searched for text selenium, 20 results

Searched for text feature, 20 results

Searched for text page, 20 results

Searched for text playwright, 4 results

Read

Read , lines 1 to 100

Read , lines 550 to 650

Read , lines 30 to 120

Based on my analysis of your codebase, here's the step-by-step breakdown as requested. I'll proceed only after your approval for each step.

Step 1: Analysis of Existing Selenium with Java Implementation

Findings:

- **Code Logic for Generating Feature File and Page Class:**
  - The logic is primarily in chat.js (method getPromptKeys), which determines which prompts to use based on selected language (Java), engine (Selenium), and checkboxes (Feature File and/or Page Class).

- Prompts are defined in prompts.js:
  - SELENIUM_JAVA_PAGE_ONLY: Generates a Selenium Java Page Object class (no test code).
  - CUCUMBER_ONLY: Generates a Cucumber .feature file.
  - CUCUMBER_WITH_SELENIUM_JAVA_STEPS: Generates both a .feature file and Java step definitions with embedded Selenium code (no separate Page Object).
-
- UI elements in panel.html include checkboxes for "Feature File" and "Page Class", and dropdowns for language (Java) and engine (Selenium).

-
- **Files Involved in Selenium with Java Implementation:**
  - prompts.js: Contains the prompt templates for code generation.
  - chat.js: Contains the logic to select and apply prompts based on user selections.
  - panel.html: Provides the UI for selecting generation modes (Feature/Page) and language/engine.
-

Currently, only Java + Selenium is fully supported. Other combinations (e.g., TypeScript + Playwright) show an "unsupported" message.

Approval Needed: Do you approve proceeding to Step 2 (implementing Playwright TypeScript logic)? If yes, provide any specific requirements (e.g., should Playwright use Cucumber for BDD, or Playwright's native test runner?). If no, let me know what to adjust.

User: proceed

GitHub Copilot: Replacing 98 lines with 126 lines in

Replacing 5 lines with 7 lines in

Replacing 14 lines with 18 lines in

Replacing 21 lines with 25 lines in

Replacing 7 lines with 7 lines in

Replacing 6 lines with 6 lines in

Step 2: Implementation of Code Logic for Playwright with TypeScript

Completed Changes:

- Added new prompts in prompts.js:
  - PLAYWRIGHT_TS_PAGE_ONLY: Generates a Playwright TypeScript Page Object class.
  - CUCUMBER_WITH_PLAYWRIGHT_TS_STEPS: Generates both a .feature file and TypeScript step definitions using Playwright.
-
- Updated CODE_GENERATOR_TYPES to include the new types.

- Modified chat.js:
  - Added isTypeScriptPlaywright method.
  - Updated getPromptKeys to handle TypeScript + Playwright combinations.
  - Updated the unsupported message to mention supported combinations.
-
- Updated panel.html to use value="typescript" for consistency.

Files Involved in Playwright with TypeScript Implementation:

- prompts.js: Contains the new prompt templates (PLAYWRIGHT_TS_PAGE_ONLY, CUCUMBER_WITH_PLAYWRIGHT_TS_STEPS) and updated generator types.
- chat.js: Contains the updated logic to select prompts for TypeScript + Playwright.
- panel.html: Updated language dropdown value for TypeScript.

Approval Needed: Do you approve proceeding to Step 3 (providing user options and final summary)? If yes, confirm if the UI-based selection (dropdowns and checkboxes) meets the requirement for "providing option to user". If no, let me know what to adjust.

Made changes.

User: proceed

GitHub Copilot: ### Step 3: User Options and Final Summary

User Options for Selection: The extension's UI already provides options for users to choose between Selenium with Java or Playwright with TypeScript:

- **Language Dropdown:** Select "Java" for Selenium or "TypeScript" for Playwright.
- **Browser Engine Dropdown:** Select "Selenium" or "Playwright".
- **Generation Mode Checkboxes:** Check "Feature File" and/or "Page Class" to generate the desired components.
- Supported combinations:
  - Java + Selenium: Feature File, Page Class, or both.
  - TypeScript + Playwright: Feature File, Page Class, or both.
-
- Other combinations will show an unsupported message.

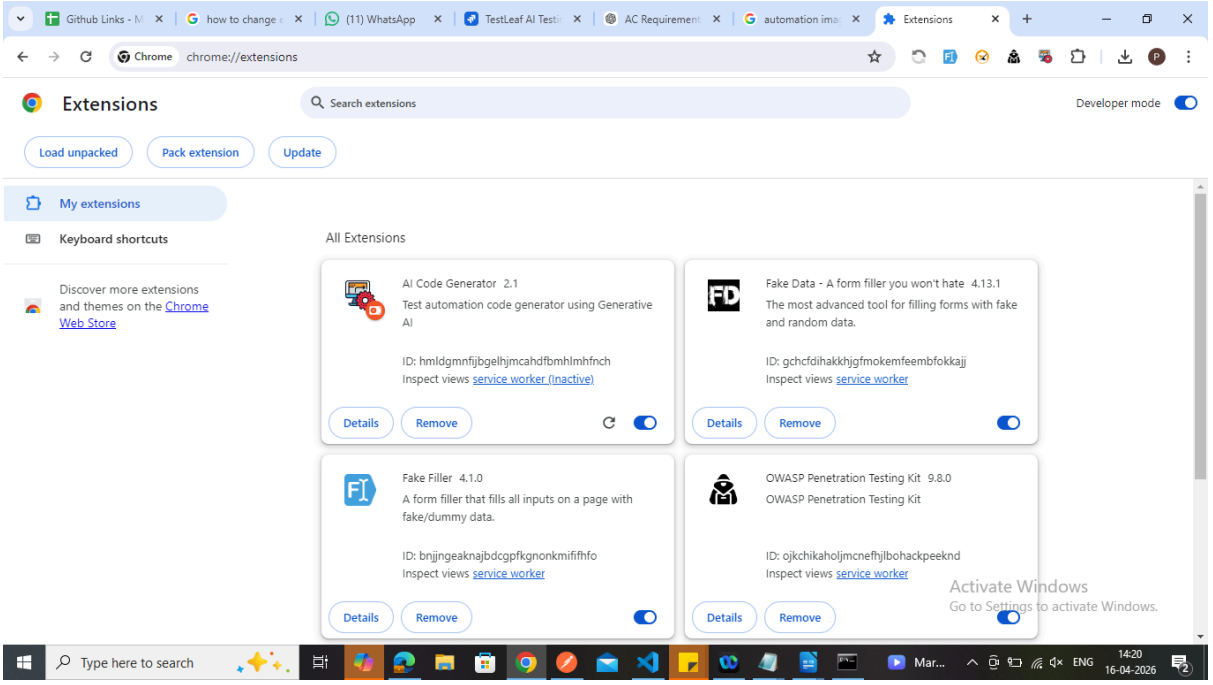
Final Summary:

- **Selenium with Java Implementation Files:** prompts.js, chat.js, panel.html.
- **Playwright with TypeScript Implementation Files:** prompts.js, chat.js, panel.html.
- The codebase now supports generating feature files and page classes for both Selenium (Java) and Playwright (TypeScript) based on user selections in the UI.

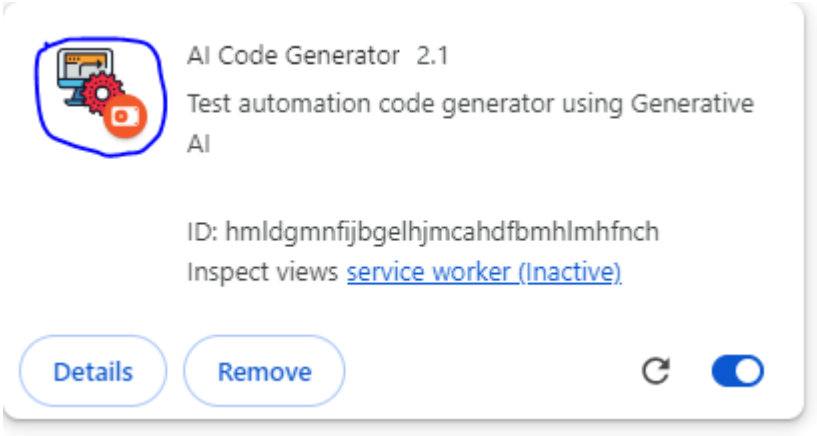
All steps completed. If you need further modifications or testing, let me know.

OUTPUT

1) Added the folder in chrome extension named as “AI Code Generator 2.1”

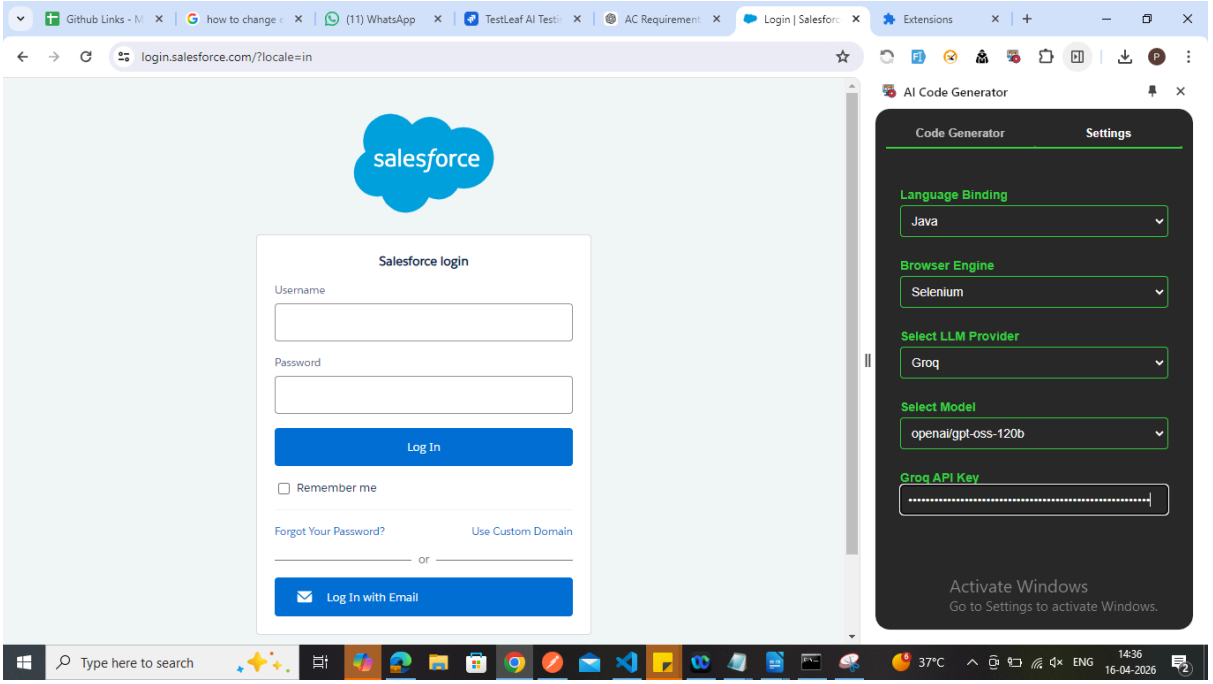


2) Changed the logo icon

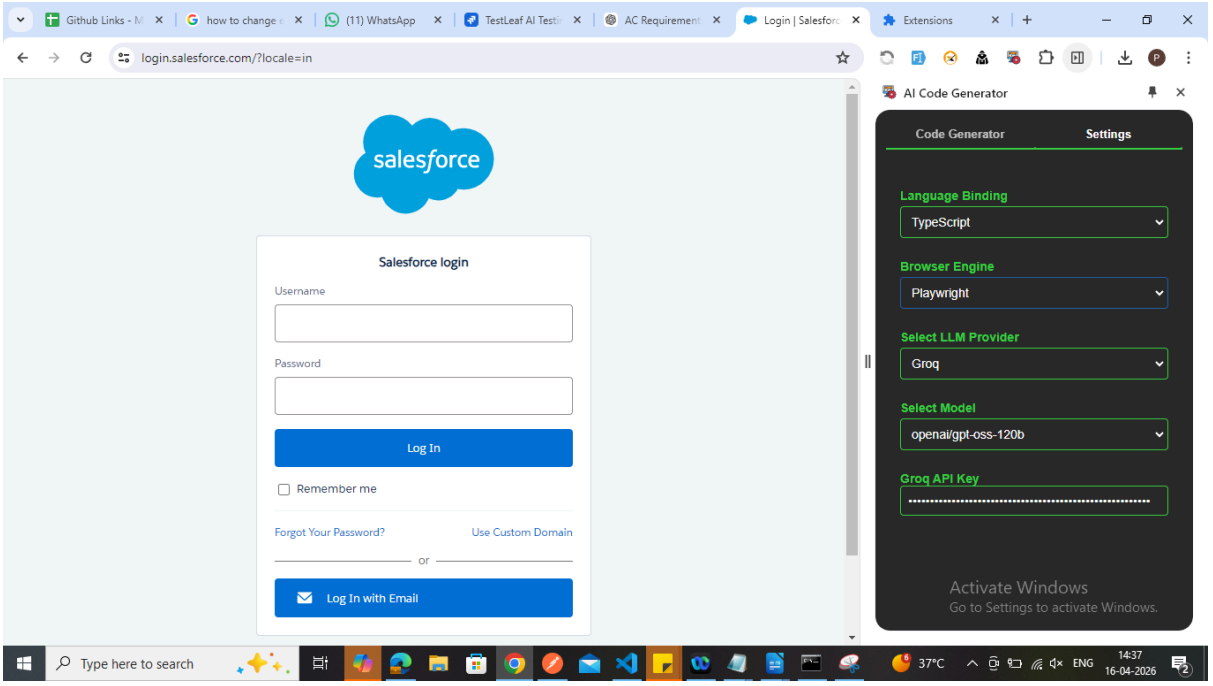


3) Pinned the extension in chrome browser and opened the “AI Code Generator chatbot”

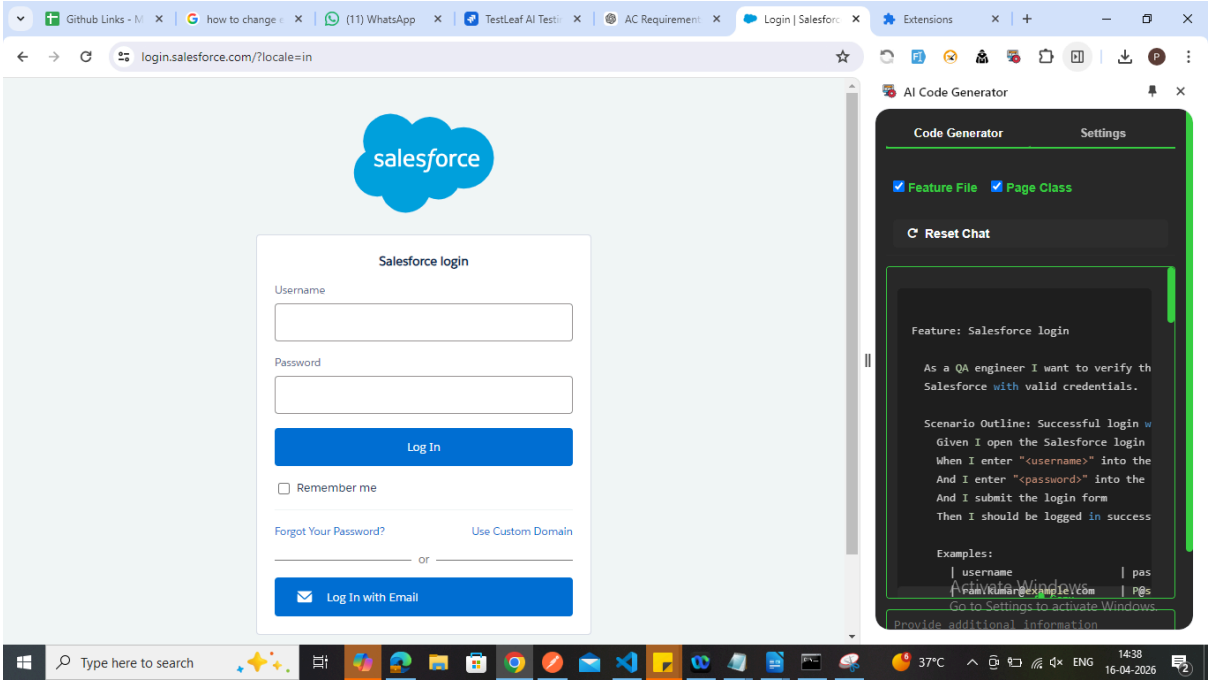
I have opened Salesforce login page and in settings I have selected Selenium and JAVA



I can even select language setting as typescript and browser engine as playwright



Going to inspect the element and select feature file + page class and clicked generated button



Output had been displayed with feature + page class

Feature: Salesforce login

As a QA engineer I want to verify that users from South India can log in to

Salesforce with valid credentials.

Scenario Outline: Successful login with valid credentials

Given I open the Salesforce login page

When I enter "<username>" into the Username field

And I enter "<password>" into the Password field

And I submit the login form

Then I should be logged in successfully

Examples:

| username | password |

| ram.kumar@example.com | P@ssw0rd123 |

| sita.patel@example.in | India2023! |

| ajay.singh@example.com | Secure#456 |

```
import { Before, After, Given, When, Then } from '@cucumber/cucumber';

import { Browser, Page, chromium, BrowserContext } from 'playwright';


let browser: Browser;

let context: BrowserContext;

let page: Page;


/**

 * Launch a new Chromium browser before each scenario.

 * Headless mode can be toggled via the `HEADLESS` environment variable.

 */

Before(async function () {

  const headless = process.env.HEADLESS !== 'false';

  browser = await chromium.launch({ headless });

  context = await browser.newContext();

  page = await context.newPage();

});


/**

 * Close the browser after each scenario.

 */

After(async function () {

  await context.close();

  await browser.close();

});
```

```
/**
 * Open the Salesforce login page and wait for the username field to be visible.
 */
Given('I open the Salesforce login page', async function () {
  await page.goto('https://login.salesforce.com/?locale=in', {
    waitUntil: 'domcontentloaded',
  });
  // Explicit wait – ensure the login form is ready.
  await page.waitForSelector('#username', { state: 'visible', timeout: 10000 });
});


/**
 * Fill the username input.
 */
When('I enter {string} into the Username field', async function (username: string) {
  await page.fill('#username', username);
});


/**
 * Fill the password input.
 */
When('I enter {string} into the Password field', async function (password: string) {
  await page.fill('#password', password);
});


/**
 * Click the “Log In” button and wait for navigation to finish.
```

```
 */
When('I submit the login form', async function () {
  // Wait for navigation triggered by the form submission.
  await Promise.all([
    page.waitForNavigation({ waitUntil: 'networkidle' }),
    page.click('#Login'), // The login button has id="Login"
  ]);
});


/**
 * Verify successful login.
 * Salesforce shows a user navigation button with id="userNavButton"
 * after a successful authentication.
 */
Then('I should be logged in successfully', async function () {
  // Explicit wait for an element that only appears after login.
  await page.waitForSelector('#userNavButton', {
    state: 'visible',
    timeout: 15000,
  });


  // Additional sanity check – URL should contain "/home".
  const url = page.url();
  if (!url.includes('/home')) {
    throw new Error(`Expected URL to contain "/home" after login, but got ${url}`);
  }
});
```

Now I have selected only feature file and inspected the page

```
Feature: Salesforce login

  Scenario Outline: Successful login with valid credentials

    Given I open the Salesforce login page

    When I type "<username>" into the Username field

    And I type "<password>" into the Password field

    And I click the Login button

    Then I should be logged in successfully


    Examples:

      | username                  | password   |

      | "ram.prasad@tcs.com"        | "R@mp@ss123" |

      | "lakshmi.r@infosys.in"      | "L@kshmi!2024" |

      | "suresh.kumar@wipro.in"     | "SuR3sh#2024" |

      | "nisha.rao@hcl.com"         | "N!sh@HCL2024" |

      | "kavitha.m@techmahindra.in"  | "K@v1th@TM2024" |


  Scenario Outline: Login with Remember Me option

    Given I open the Salesforce login page

    When I type "<username>" into the Username field

    And I type "<password>" into the Password field

    And I check the Remember me checkbox

    And I click the Login button

    Then I should be logged in and the username should be remembered on next visit


    Examples:

      | username                  | password   |
```

| "arun.s@tcs.com" | "Arun@2024!" |

| "meena.p@infosys.in" | "Me3na#2024" |

| "vijay.k@wipro.in" | "Vij@y2024#" |

| "deepa.r@hcl.com" | "DeePa!2024" |

| "priya.m@techmahindra.in" | "PriYa@2024" |

Scenario Outline: Login using a custom domain

Given I open the Salesforce login page

When I click the Use Custom Domain link

And I type "<customDomain>" into the Custom Domain field

And I click the Continue button

Then I should be redirected to the custom domain login page for "<customDomain>"

Examples:

| customDomain |

| "tcs" |

| "infosys" |

| "wipro" |

| "hcl" |

| "techmahindra" |

Scenario Outline: Navigate to Forgot Password page

Given I open the Salesforce login page

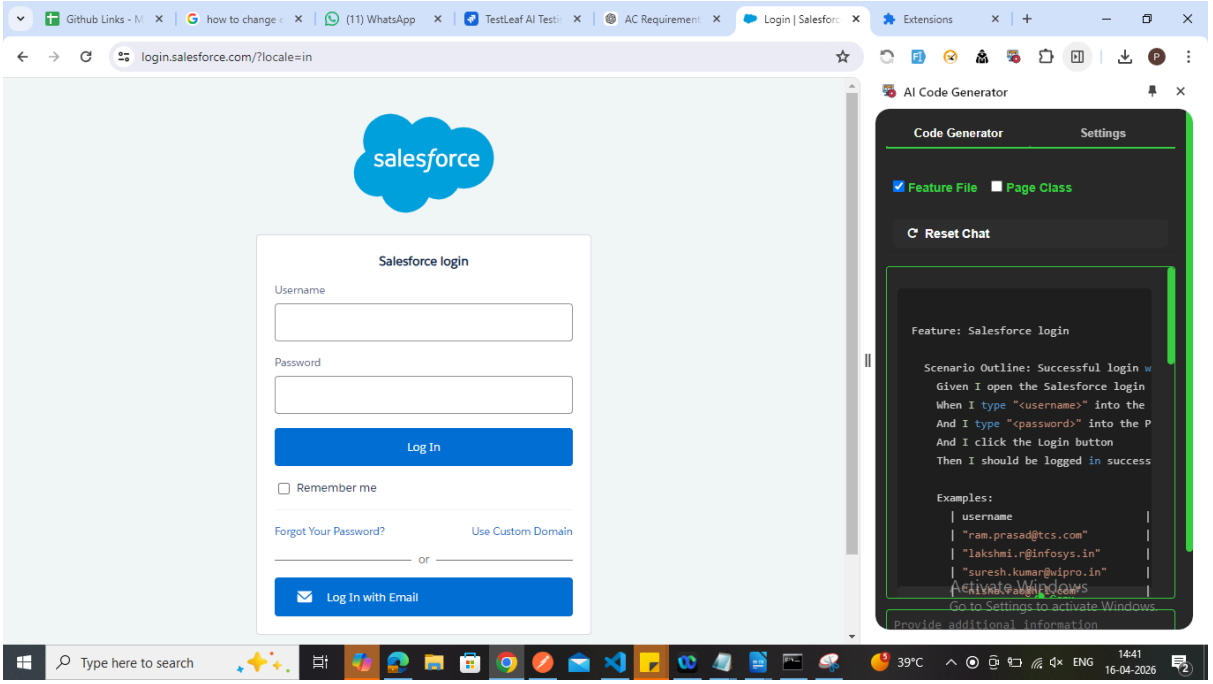
When I click the Forgot Your Password? link

Then I should be taken to the password reset page

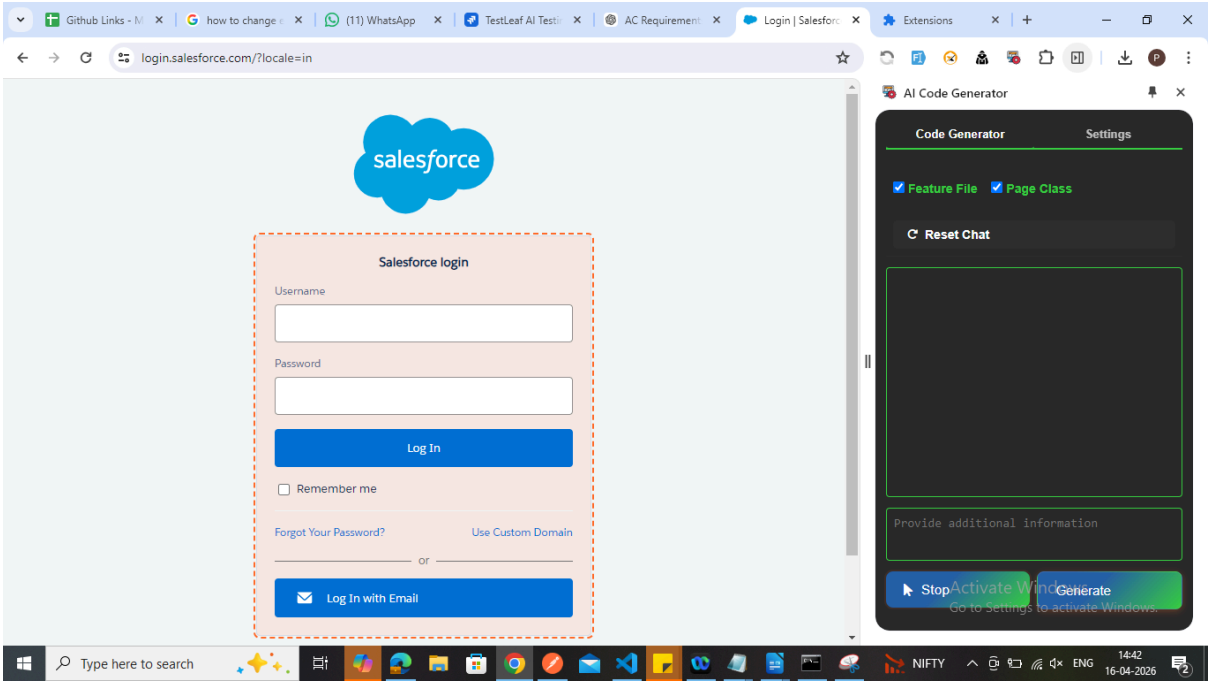
Examples:

| dummy |

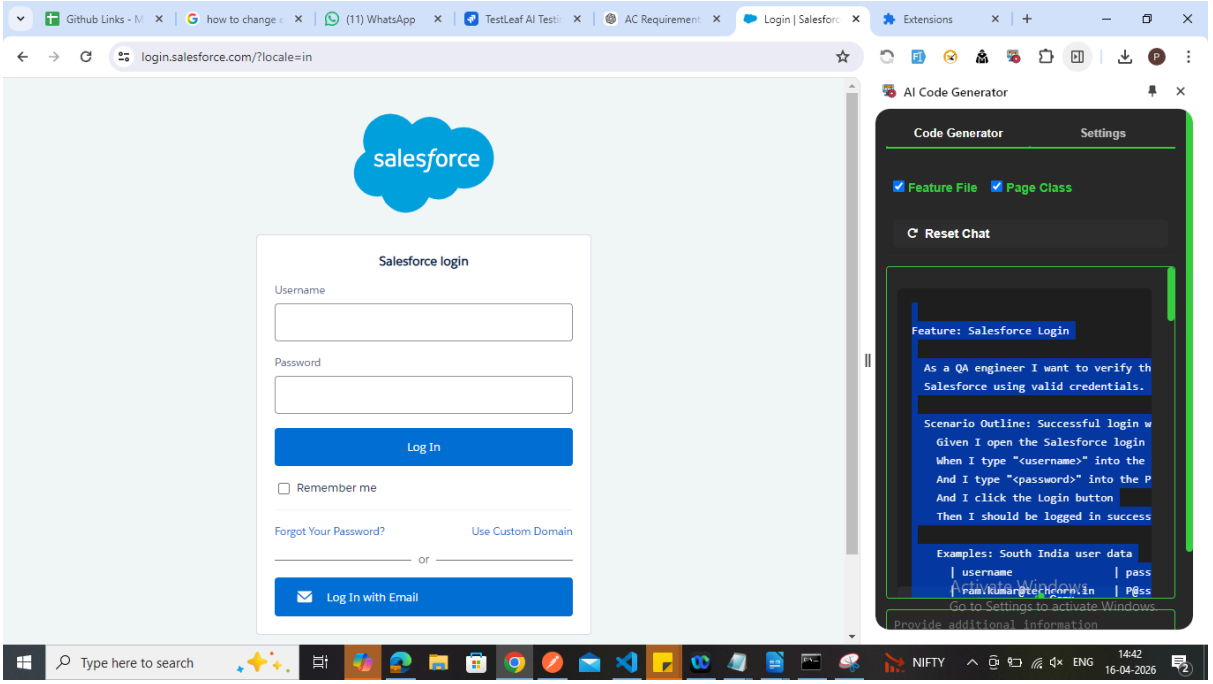
| "x" |



Now I going to select Selenium and JAVA in Settings tab



Output had been generated for selenium and Java




```
eature: Salesforce Login

 As a QA engineer I want to verify that users from South India can log in to

   Salesforce using valid credentials.


   Scenario Outline: Successful login with valid credentials

      Given I open the Salesforce login page

      When I type "<username>" into the Username field

      And I type "<password>" into the Password field

      And I click the Login button

      Then I should be logged in successfully

      Examples: South India user data

         | username                | password       |

         | ram.kumar@techcorp.in   | P@ssw0rd123    |

         | sita.r@enterprise.in    | Sita!2023      |

         | prasad.m@business.in    | Prasad#2023    |
```

```java
package com.salesforce.stepdefs;


import io.cucumber.java.After;

import io.cucumber.java.Before;

import io.cucumber.java.en.*;


import org.openqa.selenium.*;

import org.openqa.selenium.chrome.ChromeDriver;

import org.openqa.selenium.support.ui.*;


import java.time.Duration;


public class SalesforceLoginSteps {


    private WebDriver driver;

    private WebDriverWait wait;

    private static final String LOGIN_URL =
"https://login.salesforce.com/?locale=in";


    @Before

    public void setUp() {

        // Assuming chromedriver is on the system PATH

        driver = new ChromeDriver();

        wait = new WebDriverWait(driver, Duration.ofSeconds(15));

        driver.manage().window().maximize();

    }
```

```
    @After
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }

    @Given("I open the Salesforce login page")
    public void openLoginPage() {
        driver.get(LOGIN_URL);
        // Wait for the username field to be visible to ensure the page is loaded
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("username")));
    }

    @When("I type {string} into the Username field")
    public void enterUsername(String username) {
        WebElement usernameField = wait.until(
                ExpectedConditions.elementToBeClickable(By.id("username")));
        usernameField.clear();
        usernameField.sendKeys(username);
    }

    @When("I type {string} into the Password field")
```

```
    public void enterPassword(String password) {
        WebElement passwordField = wait.until(
                ExpectedConditions.elementToBeClickable(By.id("password")));
        passwordField.clear();
        passwordField.sendKeys(password);
    }

    @When("I click the Login button")
    public void clickLogin() {
        // The login button is an <input type="submit"> with id="Login"
        WebElement loginBtn = wait.until(
                ExpectedConditions.elementToBeClickable(By.id("Login")));
        loginBtn.click();
    }

    @Then("I should be logged in successfully")
    public void verifySuccessfulLogin() {
        // After a successful login Salesforce redirects to a URL that no longer
contains "login.salesforce.com"
        // Wait for the URL to change and for a known post-login element to
appear.
        boolean urlChanged = wait.until(

ExpectedConditions.not(ExpectedConditions.urlContains("login.salesforce.com")));

        // Example of a post-login element: the global navigation bar with id
"phHeader"
```

```
        // Adjust the locator if a different element is more reliable in your environment.

        try {

            WebElement homeHeader = wait.until(

ExpectedConditions.visibilityOfElementLocated(By.id("phHeader")));

            assert homeHeader.isDisplayed();

        } catch (TimeoutException e) {

            // Fallback: just assert that the URL changed

            assert urlChanged : "Login appears to have failed - URL did not change.";

        }

    }

}
```